

JavaScript Proxy

[Back to JS page](#)

Table of Contents

- [JavaScript Proxy](#)
 - [What is a Proxy?](#)
 - [Example 1](#)
 - [Proxy Logging](#)
 - [Example 2](#)
 - [Proxy Validation](#)
 - [Example 3](#)
 - [Virtual Properties](#)
 - [Example 4](#)
 - [Dynamic Functions](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

What is a Proxy?

A Proxy object wraps another object and intercepts fundamental operations like property lookup, assignment, enumeration, and function invocation.

The Proxy is created with two parameters:

- **target:** the original object that the proxy wraps
- **handler:** an object that defines which operations will be intercepted (traps)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>What is a Proxy?</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe"
};

const handler = {
  get: function(target, property) {
    return target[property];
  }
};

const proxy = new Proxy(person, handler);

document.getElementById("demo").innerHTML =
  "proxy.firstName: " + proxy.firstName + "<br>" +
  "proxy.lastName: " + proxy.lastName + "<br>" +
  "proxy is a Proxy: " + (proxy !== person);
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Proxy Logging

A Proxy can log all property accesses without modifying the original object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy Logging</h4>
<p id="demo"></p>

<script>
const user = {
  name: "Alice",
  age: 30,
  role: "admin"
};

const loggingHandler = {
  get: function(target, property) {
    console.log("LOG: Accessing property '" + property + "'");
    console.log("LOG: Current value is:", target[property]);
    return target[property];
  },
  set: function(target, property, value) {
    console.log("LOG: Setting property '" + property + "' to", value);
    console.log("LOG: Previous value was:", target[property]);
    target[property] = value;
    return true;
  }
};

const proxy = new Proxy(user, loggingHandler);

// Access and modify through the proxy
proxy.name = "Bob";
let userRole = proxy.role;
proxy.age = 31;

document.getElementById("demo").innerHTML =
  "name: " + proxy.name + "<br>" +
  "age: " + proxy.age + "<br>" +
  "role: " + proxy.role + "<br><br>" +
  "(Check browser console to see all log messages)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Proxy Validation

A Proxy can validate data before setting it on the target object:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Proxy Validation</h4>
<p id="demo"></p>

<script>
const person = {
  name: "John",
  age: 25
};

const validator = {
  set: function(target, property, value) {
    if (property === "age") {
      if (typeof value !== "number") {
        throw new Error("Age must be a number");
      }
      if (value < 0 || value > 120) {
        throw new Error("Age must be between 0 and 120");
      }
    }
    if (property === "name") {
      if (typeof value !== "string") {
        throw new Error("Name must be a string");
      }
      if (value.length < 2) {
        throw new Error("Name must be at least 2 characters");
      }
    }
    target[property] = value;
    return true;
  }
};

const proxy = new Proxy(person, validator);

try {
  proxy.age = 30;    // Valid
  proxy.name = "Jane"; // Valid
  // proxy.age = 200; // Would throw error
  // proxy.name = "J"; // Would throw error

  document.getElementById("demo").innerHTML =
    "Validated proxy:<br>" +
    "name: " + proxy.name + "<br>" +

```

```
    "age: " + proxy.age + "<br><br>" +  
    "Age and name validation is active – invalid values are rejected!";  
  } catch(err) {  
    document.getElementById("demo").innerHTML = "Error: " + err.message;  
  }  
</script>  
  
</body>  
</html>
```

Example 3

Result [View Example](#)

Virtual Properties

A Proxy can create virtual properties that don't exist on the target object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Virtual Properties</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  birthYear: 1990
};

const handler = {
  get: function(target, property) {
    if (property === "fullName") {
      return target.firstName + " " + target.lastName;
    }
    if (property === "age") {
      const year = new Date().getFullYear();
      return year - target.birthYear;
    }
    return target[property];
  }
};

const proxy = new Proxy(person, handler);

document.getElementById("demo").innerHTML =
  "firstName: " + proxy.firstName + "<br>" +
  "lastName: " + proxy.lastName + "<br>" +
  "fullName (virtual): " + proxy.fullName + "<br>" +
  "age (virtual): " + proxy.age;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Dynamic Functions

A Proxy can intercept function calls using the `apply` trap:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Proxy</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Dynamic Functions</h4>
<p id="demo"></p>

<script>
function greet(name) {
  return "Hello, " + name + "!";
}

const handler = {
  apply: function(target, thisArg, args) {
    console.log("Function called with arguments:", args);
    console.log("Call count: " + (++callCount));
    return target.apply(thisArg, args);
  }
};

let callCount = 0;
const proxy = new Proxy(greet, handler);

// Call through proxy
let result1 = proxy("Alice");
let result2 = proxy("Bob");
let result3 = proxy("Charlie");

document.getElementById("demo").innerHTML =
  result1 + "<br>" +
  result2 + "<br>" +
  result3 + "<br><br>" +
  "Total function calls: " + callCount + "<br>" +
  "(Check browser console for call logs)";
</script>

</body>
</html>
```


Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Proxy](#)